Aasmund Gabestad Nørsett

Delayed Weight Updates for High Throughput Hardware Accelerator of the CCSDS-123.0-B-2 Compression Algorithm

Master’s thesis in Electronic System Design and Innovation
Supervisor: Milica Orlandic
Co-supervisor: Samuel Boyle
June 2026

Norwegian University of Science and Technology
Faculty of Information Technology and Electrical Engineering
Department of Electronic Systems

NTNU

Abstract

Sammendrag

AI Declaration

Contents

[1]The section on hardware accelerators is taken as is from project thesis (TODO: make proper cite)

Acronyms

ADC Analog-to-Digital Converter. 15

ASIC Application Specific Integrated Circuit. 14

BI Band Interleaved. 3

BIL Band Interleaved by Line. 3, 4, 11, 18

BIP Band Interleaved by Pixel. 3, 4, 11, 18

BRAM Block Random Access Memory. 14

BSQ Band Sequential. 3, 11

CCSDS Consultative Committee for Space Data Systems. 2

CLB Configurable Logic Block. 14

COTS Component off-the-Shelf. 15

CPU Central Processing Unit. 13–15

DMA Direct Memory Access. 15

DPCM Differential Pulse Code Modulation. 2

DSP Digital Signal Processor. 14

FF Flip-Flop. 14

FL Fast Lossless. 2

FLEX Fast Lossless Extended. 2

FPGA Field-Programmable Gate Array. 1, 2, 14, 15

GPO2 Golomb power-of-two. 13

GPU Graphical Processing Unit. 14, 15

HLS High-Level Synthesis. 15

HYPSO HYPerspectral Smallsat for Ocean observation. 1, 2, 15

LUT Look-Up-Table. 14

MPSoC Multiprocessor System-on-Chip. 15

MSE Mean Square Error. 6

NTNU Norwegian University of Science and Technology. 1

PE Processing Element. 13

PL Programmable Logic. 15

PnR Place and Route. 14

RTL Register Transfer Level. 14, 15

V2V Variable-to-Variable. 12, 13

Chapter 1

Introduction

With the development of small satellite technologies such as cube satellites (cubesats), with lower costs and shorter development times, the availability of remote sensing data has increased. Hereunder, hyperspectral and multispectral imagers provide data that is vital for ocean monitoring, geological research and observation of other natural processes [1, 2].

As part of the HYPerspectral Smallsat for Ocean observation (HYPSO) project at the Norwegian University of Science and Technology (NTNU), two 6U cubesats, HYPSO-1 and HYPSO-2, were launched in 2022 and 2024 respectively. These carry hyperspectral imagers capable of sensing wavelengths ranging from 430 nm to 800 nm over 120 spectral bands. Their primary purpose is to monitor algal blooms in the ocean, which can give indications as to the health of marine environments [3]. This is made possible by the reflection of characteristic wavelengths from chlorophyll, detectable by the on-board hyperspectral imagers.

A nominal HYPSO image has a size of roughly 150 MB. This presents a challenge due to limited on-board resources for storing large amounts of data, and low downlink speeds. As an example, HYPSO-1 is restricted to 75 MB of downlinked data per orbital revolution [3]. This prompts the need for on-board image compression to increase the scientific output and value of the mission.

Small satellites operate under strict limitations on on-board resources such as processor time and power budgets. This, combined with the complexity of image compression algorithms, makes the use of custom hardware accelerators ideal. For their implementation, Field-Programmable Gate Array (FPGA)s are the perfect choice, providing great flexibility and speed at a low cost.

This thesis explores the use of a novel weight update scheme proposed by Jia et al. for the CCSDS-123.0-B-2 compression algorithm. The purpose is to increase the throughput of the accelerator implemented by [5] for use on-board the HYPSO missions. This should not come at significant degradation in compression performance or hardware resource utilization. Additionally, an open-source decompressor has been implemented in order to verify the results of the proposed change in the algorithm.

The rest of the paper is organized as follows...

Chapter 2

Background

2.1 Hyperspectral Image Compression

The Consultative Committee for Space Data Systems (CCSDS) is a multi-institutional working group developing algorithms and standards for increased cooperation amongst space actors. A portion of their work has been concerned with the effective encoding of hyperspectral and multispectral images. Standard CCSDS 123.0-B-1, hereby referred to as issue 1, describes a lossless compression algorithm tailored for high throughput FPGA implementations. Issue 1 has been implemented and used successfully by several missions, the HYPSO project being one of them [1, 6]. In 2019, Issue 1 was followed by standard CCSDS 123.0-B-2, referred to as Issue 2. The following sections are based on the description of issue 2 presented in The Consultative Committee for Space Data Systems, 'Low-Complexity Lossless and Near-Lossless Multispectral and Hyperspectral Image Compression,' CCSDS, Recommended Standard CCSDS 123.0-B-2, 2019.

Issue 1 formalizes the Fast Lossless (FL) algorithm presented by Klimesh, which uses an adaptive filtering prediction technique to achieve lossless compression at low computational complexity suitable for space applications [8, 9]. To meet the needs of limited satellite downlink capabilities, lossy compression techniques have seen increasing use [9]. For this reason, issue 2 introduces a new near-lossless feature based on the Fast Lossless Extended (FLEX) algorithm presented by Keymeulen et al. [10]. Near-lossless compression, as opposed to lossy compression, provides the user with fine-tuned control over the per-pixel loss through tunable error limit parameters.

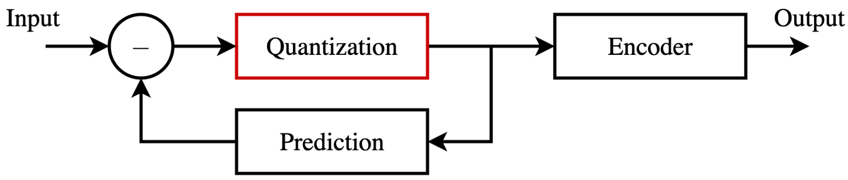


Figure 2.1: Layout of a differential pulse code modulator. The prediction loop, with optional quantization, reduces the entropy in the data passed to the encoder.

The issue 1 and issue 2 compressors use a form of Differential Pulse Code Modulation (DPCM), presented in Figure 2.1. A feedback loop, with optional quantization, predicts the value of the current input sample based on previously processed samples. The difference between the sample and the prediction, called the prediction residual, is encoded in the final bitstream. This exploits the correlation between neighbouring samples inherent in image data to reduce the entropy of the data passed to the encoder, allowing a more efficient encoding. Samples are processed in a single pass using simple operations. This reduces memory requirements and makes the algorithm suitable for hardware implementations. For the rest of this document, the prediction feedback loop will be referred to as the predictor.

Predictor outputs are passed to an entropy coder which losslessly encodes prediction residuals in the final bitstream. Entropy coders, which will be covered in greater detail later in this document, encode samples with a varying number of bits based on the frequency of their occurrence in the image. A novel feature of issue 2 is the inclusion of a hybrid encoder, in addition to the sample-adaptive and block-adaptive encoders of issue 1. This performs better at the low encoding bitrates achievable in the near-lossless mode. After encoding, a header containing image metadata and compressor configuration parameters is prepended to the compressed bitstream (image body). This is required by the decompressor in order to correctly reconstruct the original image.

The predictor uses an adaptive linear filter method based on the work presented by Gersho [9, 11]. Observed quantities x_k, analogous to the quantized prediction residuals of Figure 2.1, which are statistically dependant on the process d_k, analogous to the image samples, produce estimates z_k of the desired samples d_k. A linear preprocessor ϕ takes previously observed quantities x_k and transforms them to a vector $\mathbf{u}_k$ given by

$$\mathbf{u}_k = \Phi(x_k, x_{k-1}, \ldots). \tag{2.1}$$

The elements of $\mathbf{u}_k$ are weighted by a vector $\mathbf{c}_k$ to produce the prediction z_k as

$$z_k = \mathbf{c}_k \cdot \mathbf{u}_k. \tag{2.2}$$

The weight vector is adapted (updated) in increments $\Delta\mathbf{c}_k$ based on observed error residuals $z_k - d_k$ of previous predictions.

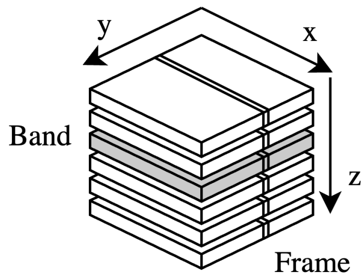


Figure 2.2: Layout of a hyperspectral image. Samples of the same z coordinates belong to the same frequency band, whereas samples of the same y coordinate are denoted as frames. Samples within the same frame of a band are denoted as lines.

Figure 2.2 shows the layout of a hyperspectral image. The sizes of the dimensions x, y, and z are denoted N_x, N_y, and N_z. Samples of the same z coordinate correspond to frequency bands, while samples of the same y coordinate are denoted as part of the same frame. In a bushbroom imager, like the one on-board the HYPSO-1 mission, images are captured one frame at a time [3]. As the satellite moves, subsequent frames are concatenated to form the final image. The camera frame rate controls the time, and thus physical space, between captured frames. Samples within the same frame of a band are referred to as lines, and samples of the same x and y coordinate belong to a pixel.

In order to correctly reproduce the original image, the sample processing order of the compressor and decompressor must be the same. The standard recognizes two main categories of processing orders: Band Sequential (BSQ) and Band Interleaved (BI). BSQ ordering processes samples in coordinate order (x, y, z), meaning all samples of one band are processed before proceeding to the next. BI ordering processes samples frame by frame. The frame interleaving depth M dictates the processing order within the frame. The standard defines two special cases: Band Interleaved by Line (BIL) and Band Interleaved by Pixel (BIP), with interleaving depths equal to 1 and N_z respectively. BIL processes

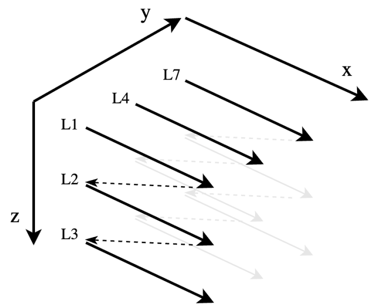


Figure 2.3: BIL processing order. Lines, L1 to L3, within the first frame are processed before proceeding to the next frame.

samples within a frame line by line in coordinate order (x, z, y), as presented by Figure 2.3. BIP processes samples in the order (z, x, y).

Image samples may be referred to by their x, y, and z coordinates as $s(x, y, z)$. To shorten the notation, the standard introduces the spatial index t, given by $t = yN_x + x$. Thus, a sample may be referred to as $s_z(t)$. The number of bits used to represent a sample is the dynamic range D.

The following sections give a brief technical overview of the predictor and encoder stages of the issue 2 algorithm. Symbols are mentioned in text, and are the same as those used in the issue 2 standard unless otherwise is specified. As a quick reference, and for the readers convenience, all symbols are listed in Table 2.1. For a detailed review of the standard please see The Consultative Committee for Space Data Systems, 'Low-Complexity Lossless and Near-Lossless Multispectral and Hyperspectral Image Compression,' CCSDS, Recommended Standard CCSDS 123.0-B-2, 2019.

Table 2.1: Issue 2 compressor symbol table.

Symbol	Name
$s_z(t)$	Image sample
$\sigma_z(t)$	Local sum
$\mathbf{U}_z(t)$	Local difference vector
$d_z(t)$	Central local difference
$\hat{d}_z(t)$	Predicted central local difference
$\breve{s}_z(t)$	High-resolution predicted sample
$\tilde{s}_z(t)$	Double-resolution predicted sample
$\hat{s}_z(t)$	Predicted sample
$\Delta_z(t)$	Prediction residual
$m_z(t)$	Maximum error
$q_z(t)$	Signed quantizer index
$s'_z(t)$	Clipped quantizer bin center
$\tilde{s}''_z(t)$	Double-resolution sample representative
$s''_z(t)$	Sample representative
$\theta_z(t)$	Predicted sample and sample endpoint difference
$\delta_z(t)$	Mapped quantizer index
$e_z(t)$	Double-resolution prediction error
$\mathbf{W}_z(t)$	Weight vector
$\tilde{\Sigma}_z(t)$	High-resolution accumulator
$\Gamma(t)$	Counter

2.1.1 Predictor

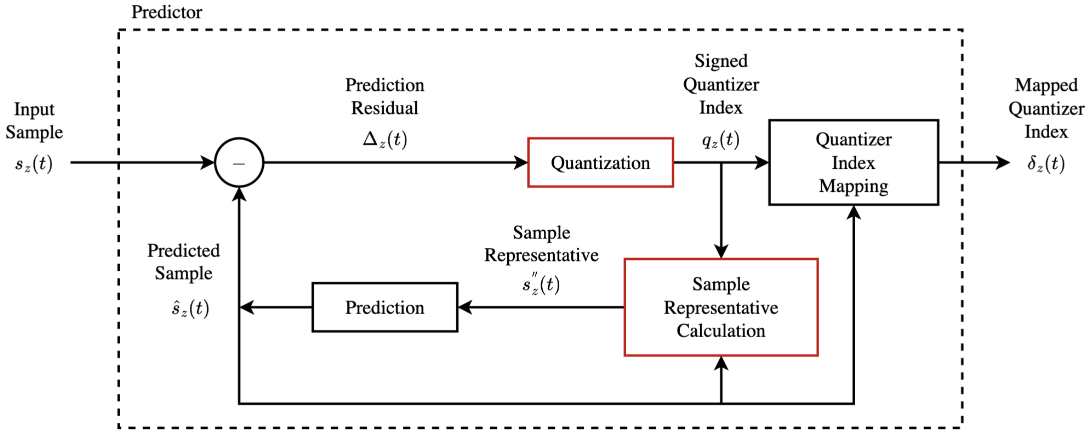


Figure 2.4: Block diagram of the predictor stage of issue 2. The quantization and sample representative steps, marked in red, are new inclusions compared to Issue 1.

Figure 2.4 shows a block diagram of the issue 2 predictor stage with new features as compared to issue 1 marked in red. In the decompressor, the outputs of the predictor stage are first losslessly reproduced by decoding the encoder outputs stored in the compressed bitstream. In order to correctly retrieve the original image, the decompressor must be able to reproduce the prediction model of the compressor. Normally, this would require the samples from the original image to be known. To solve this, the issue 2 predictor makes use of sample representatives $s''_z(t)$ which may be calculated directly from the quantized prediction residuals, and thus available when decompressing.

Quantization

Prediction residuals $\Delta_z(t)$ are quantized by a uniform quantizer producing quantizer indices $q_z(t)$ given by

$$q_z(t) = \begin{cases} \Delta_z(0), & t = 0, \\ \operatorname{sgn}(\Delta_z(t)) \left\lfloor \dfrac{|\Delta_z(t)| + m_z(t)}{2m_z(t)+1} \right\rfloor, & t > 0, \end{cases} \tag{2.3}$$

where the prediction residual is the difference between input samples $s_z(t)$ and predicted sample values $\hat{s}_z(t)$ given by

$$\Delta_z(t) = s_z(t) - \hat{s}_z(t). \tag{2.4}$$

Figure 2.5 shows a uniform quantizer centered at zero. Inputs are mapped to bins with size Δ, also called the step size [12]. The step size of the issue 2 quantizer is $2m_z(t)+1$ and is controlled by the maximum error value $m_z(t)$. This guaranties that the prediction residual can be reconstructed with an error of at most $\pm m_z(t)$. A quantizer index of zero means that the predicted sample was within $m_z(t)$ of the actual sample. Positive and negative quantizer indices indicate that the prediction overshot or undershot the sample, respectively. The clipped quantizer bin center $s'_z(t)$ is used for calculation of sample representatives, and is given by

$$s'_z(t) = \operatorname{clip}\big(\hat{s}_z(t) + q_z(t)\big(2m_z(t)+1\big),\ \{s_{\min}, s_{\max}\}\big), \tag{2.5}$$

where s_{min} and s_{max} are the lower and upper bounds for the input sample value dictated by the dynamic range D.

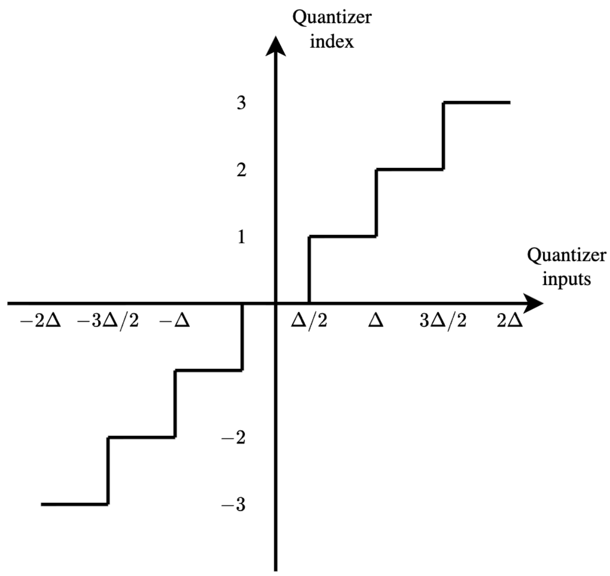


Figure 2.5: Uniform quantizer with step size Δ.

The user may select an absolute and a relative error limit, a_z and r_z respectively, giving fine tuned control over the per pixel difference as compared to the original image. To losslessly encode image samples, the maximum error limit may be set to zero. When error limits are used, the maximum error is given by

$$m_z(t) = \min\left(a_z, \left\lfloor \frac{|r_z \hat{s}_z(t)|}{2^D} \right\rfloor\right). \tag{2.6}$$

Because the codewords used by the entropy coder to produce the compressed bitstream are only defined for positive integers, the signed quantizer indices are mapped to positive integer mapped quantizer indices $\delta_z(t)$. This mapping is reversible and is given by

$$\delta_z(t) = \begin{cases} |q_z(t)| + \theta_z(t), & |q_z(t)| > \theta_z(t), \\ 2|q_z(t)|, & 0 \leq (-1)^{\tilde{s}_z(t)} q_z(t) \leq \theta_z(t), \\ 2|q_z(t)| - 1, & \text{otherwise}, \end{cases} \tag{2.7}$$

where the double-resolution predicted sample value $\tilde{s}_z(t)$ is an intermediate value produced by the prediction stage (Figure 2.4) which will be covered later in the document. The predicted sample and sample endpoint difference $\theta_z(t)$ is given by

$$\theta_z(t) = \begin{cases} \min\{\hat{s}_z(0) - s_{\min},\ s_{\max} - \hat{s}_z(0)\}, & t = 0, \\ \min\left(\left\lfloor \frac{|\hat{s}_z(t) - s_{\min} + m_z(t)|}{2m_z(t)+1} \right\rfloor, \left\lfloor \frac{|s_{\max} - \hat{s}_z(t) + m_z(t)|}{2m_z(t)+1} \right\rfloor\right), & t > 0. \end{cases} \tag{2.8}$$

Sample representatives

As mentioned, sample representatives are used in place of the sample values in order to allow the decompressor to reproduce the prediction model. Selecting sample representatives to be equal to the quantizer bin center minimizes the per-pixel reconstruction error. However, a different selection may be used to minimize other quality metrics, such as the Mean Square Error (MSE) (section 2.3) [9]. For this reason, the user is provided with a set of parameters which control the selection of the sample representatives in relation to the quantizer bin center. Intermediary values of the sample representative calculation are the double-resolution sample representative $\tilde{s}''_z(t)$ and the quantizer bin center $s'_z(t)$.

Local Sums and Differences

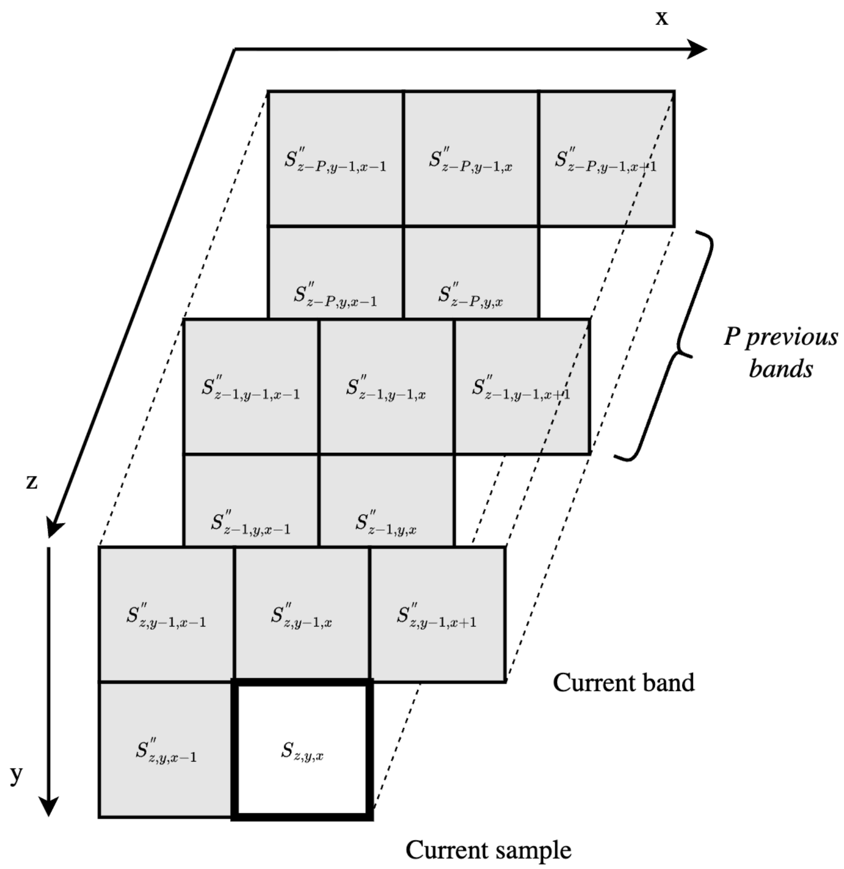


Figure 2.6: A typical prediction neighborhood. Local sums are used to find local differences in P previous bands (prediction bands) which are used to find the predicted sample.

Sample representatives of previously processed samples form the prediction neighborhood as shown in Figure 2.6. The parameter P controls the number of spectral bands that are included in the prediction calculation, called prediction bands. A weighted sum of spatially adjacent samples to the current sample form the local sum $\sigma_z(t)$. Issue 2 defines two categories of local sum configurations: column-oriented and neighbor-oriented, both of which can be configured as wide and narrow. Figure 2.7 shows the wide and narrow configurations for neighbor-oriented local sums. Full details and corner cases of the local sum configurations are not important for this discussion, but as an example, narrow neighbor-oriented local sums are defined as

$$\sigma_{z,y,x} = \begin{cases} s''_{z,y-1,x-1} + 2s''_{z,y-1,x} + s''_{z,y-1,x+1}, & y>0,\ 0<x<N_X-1, \\ 4s''_{z-1,y,x-1}, & y=0,\ x>0,\ z>0, \\ 2\left(s''_{z,y-1,x} + s''_{z,y-1,x+1}\right), & y>0,\ x=0, \\ 2\left(s''_{z,y-1,x-1} + s''_{z,y-1,x}\right), & y>0,\ x=N_X-1, \\ 4s_{\mathrm{mid}}, & y=0,\ x=0,\ z=0. \end{cases} \tag{2.9}$$

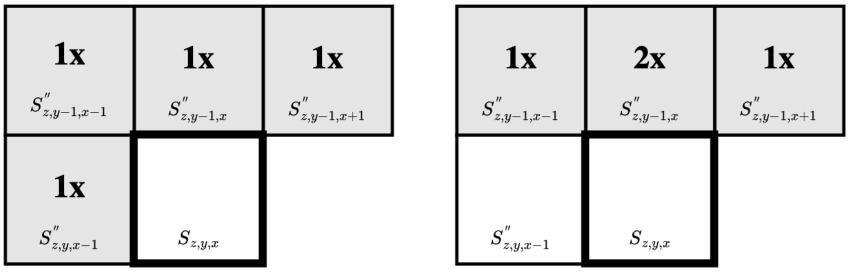


Figure 2.7: Wide and narrow neighbor-oriented local sum configurations.

The (central) local difference $d_z(t)$ is the difference between the local sum and the current sample. Local differences of the prediction bands form the local difference vector $\mathbf{U}_z(t)$, and is analogous to the vector $\mathbf{u}_k$ (Equation 2.1) of the adaptive linear filter. The vector is defined as

$$\mathbf{U}_z(t) = \begin{bmatrix} d_{z-1}(t) \\ d_{z-2}(t) \\ \vdots \\ d_{z-P_z^*}(t) \end{bmatrix}, \tag{2.10}$$

where P_z^* is the number of used prediction bands found by

$$P_z^* = \min(z, P). \tag{2.11}$$

Two different prediction modes are defined in the standard: full and reduced. The above definition of the local difference vector is that of the reduced mode. During full prediction mode, three additional local difference values $d_z^N(t)$, $d_z^W(t)$ and $d_z^{NW}(t)$ are added to the vector, named directional local differences. These are calculated as the weighted difference between sample representatives in three directions and the local sum, and add avoidable dependencies on neighbouring sample representatives. For the full definition see the standard.

Weight Updates

The predicted central local difference $\hat{d}_z(t)$ is computed as

$$\hat{d}_z(t) = \mathbf{W}_z(t) \cdot \mathbf{U}_z(t). \tag{2.12}$$

Here $\mathbf{W}_z(t)$ is the weight vector, which for reduced prediction mode is defined as

$$\mathbf{W}_z(t) = \begin{bmatrix} \omega_z^{(1)}(t) \\ \omega_z^{(2)}(t) \\ \vdots \\ \omega_z^{(P_z^*)}(t) \end{bmatrix}, \tag{2.13}$$

where $\omega_z^{(i)}(t)$ is the weight corresponding to the ith prediction band. As with the local difference vector, three additional weights are used during full prediction mode.

Weights may be initialized using two methods: default and custom initiaization. Custom initialization allows the user to the provide per-band initial weight values. This may result in better compression performance when the image sensor characteristics or other image metrics are know. For default initiaization the weights are initialized as

$$\omega_z^{(1)}(1) = \frac{7}{8}2^{\Omega}, \qquad \omega_z^{(i)}(1) = \left\lfloor \frac{1}{8}\,\omega_z^{(i-1)}(1) \right\rfloor, \quad i = 2, 3, \ldots, P_z^*. \tag{2.14}$$

Weights are represented as signed integers of $\Omega + 3$ bits, where Ω is the weight bit resolution. This gives a range of possibles weight values between ω_{min} and ω_{max} as

$$\omega_{\min} = -2^{\Omega+2},\ \omega_{\max} = -2^{\Omega+2} - 1. \tag{2.15}$$

Weights are updated for every sample based on the performance of the prediction model. Using the notation presented by Jia et al. in [4] we define the weight offset as

$$\omega_{z,\text{offset}}^{(i)}(t) = \left\lfloor \frac{1}{2}\left(2^{-\rho(t)} \cdot d_{z-i}(t) \cdot \text{sgn}^+[e_z(t)] + 1\right) \right\rfloor, \tag{2.16}$$

where the double-resolution prediction error $e_z(t)$ is found using the quantizer bin center and double-resolution predicted sample value as

$$e_z(t) = 2s_z'(t) - \tilde{s}_z(t). \tag{2.17}$$

The user can control how quickly the prediction model adapts to rapid changes in the sample values by use of the weight update scaling exponent, given by

$$\rho(t) = \text{clip}\left(\nu_{\min} + \left\lfloor \frac{t - N_x}{t_{\text{inc}}} \right\rfloor, \{\nu_{\min}, \nu_{\max}\}\right) + D - \Omega, \tag{2.18}$$

where ν_{min}, ν_{max}, and t_{inc} are user tunable parameters. The final updated weight is calculated as

$$\omega_z^{(i)}(t+1) = \text{clip}\left(\omega_z^{(i)}(t) + \omega_{z,\text{offset}}^{(i)}(t), \{\omega_{min}, \omega_{max}\}\right). \tag{2.19}$$

Prediction Calculation

The predicted central local difference $\hat{d}_z(t)$ in combination with the local sum $\sigma_z(t)$ produces the high-resolution predicted sample value $\breve{s}_z(t)$ as

$$\breve{s}_z(t) = \text{clip}\left(\begin{array}{l} \text{mod}_R^*\left[\hat{d}_z(t) + 2^{\Omega}\left(\sigma_z(t) - 4s_{\text{mid}}\right)\right] \\ \quad + 2^{\Omega+2}s_{\text{mid}} + 2^{\Omega+1},\ \left\{2^{\Omega+2}s_{\text{min}},\ 2^{\Omega+2}s_{\text{max}} + 2^{\Omega+1}\right\} \end{array}\right). \tag{2.20}$$

where R is a user tunable parameter and s_{mid} is the middle sample value, dictated by the dynamic range D. From this, the double-resolution predicted sample value $\tilde{s}_z(t)$ is obtained as

$$\tilde{s}_z(t) = \begin{cases} \left\lfloor \dfrac{\breve{s}_z(t)}{2^{\Omega+1}} \right\rfloor, & t > 0, \\ 2s_{z-1}(t), & t = 0,\ P > 0,\ z > 0, \\ 2s_{\text{mid}}, & t = 0 \text{ and } (P = 0 \text{ or } z = 0). \end{cases} \tag{2.21}$$

Finally, the predicted sample value $\hat{s}_z(t)$ is given by

$$\hat{s}_z(t) = \left\lfloor \frac{\tilde{s}_z(t)}{2} \right\rfloor. \tag{2.22}$$

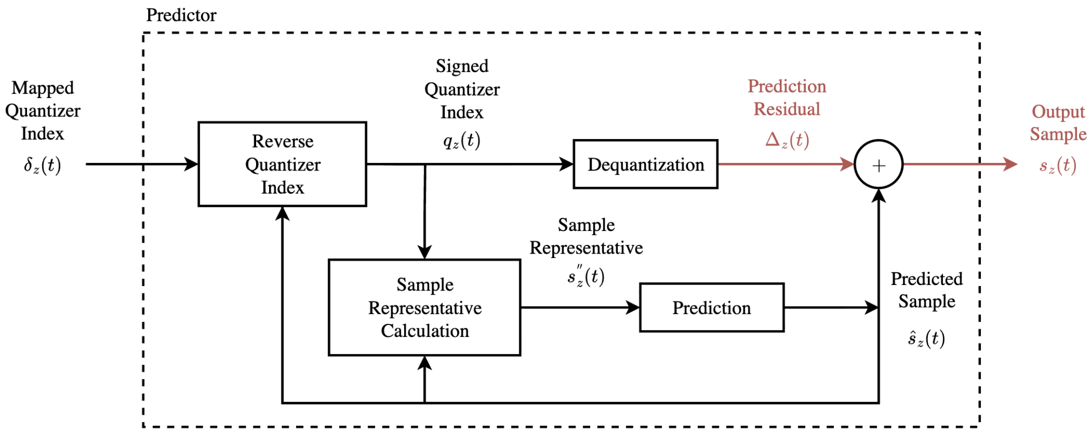


Figure 2.8: Block diagram of the predictor stage of the decompressor. The prediction model is identical to that of the compressor, except for the prediction residual and reconstructed sample during near-lossless mode, marked in red.

Decompression

Figure 2.8 shows a block diagram of the predictor stage of the decompressor, which replicates the prediction model of the compressor. Quantizer indices are reconstructed without loss from the mapped quantizer indices as

$$q_z(t) = \begin{cases} (\theta_z(t) - \delta_z(t)) \operatorname{sgn}^+\!\left(\hat{s}_z(t) - s_{\text{mid}}\right), & \delta_z(t) > 2\theta_z(t), \\ \left\lfloor \dfrac{\delta_z(t)+1}{2} \right\rfloor (-1)^{\tilde{s}_z(t)+\delta_z(t)}, & \delta_z(t) \leq 2\theta_z(t), \end{cases} \tag{2.23}$$

where $\hat{s}_z(t)$ and $\theta_z(t)$ are computed using prediction data from previously decoded samples. The range of possible values for the reconstructed sample is now $[s'_z(t) - m_z(t), s'_z(t) + m_z(t)]$ [9]. When the maximum error $m_z(t)$ is zero, prediction residuals may be reconstructed without loss. For this work the quantizer bin center $s'_z(t)$ is selected for reconstruction of samples. Thus, the dequantization step is given by

$$\Delta_z(t) = \begin{cases} q_z(0), & t = 0, \\ \operatorname{sgn}(q_z(t))|q_z(t)|(2m_z(t)+1), & t > 0. \end{cases} \tag{2.24}$$

Finally, the reconstructed sample is retrieved from the predicted sample value and dequantized prediction residual as

$$s_z(t) = \operatorname{clip}(\Delta_z(t) + \hat{s}_z(t),\ \{s_{\text{min}}, s_{\text{max}}\}). \tag{2.25}$$

Data Dependencies

Both the issue 1 and issue 2 algorithms contain backwards data dependencies that make efficient implementations for hardware challenging. Dependencies are split into two groups: spectral and spatial dependencies, referred to as $z-1$ and $t-1$ dependencies. As an example take the calculation of the weight vector, which is present in both versions of the algorithm, and for issue 2 is given by equations 2.16 and 2.19. This involves a $t-1$ dependency on the weight vector of sample $s_z(t-1)$. Pipelined hardware implementation must wait for such dependencies to resolve before proceeding, placing a restriction on the maximum achievable throughput.

The selection of sample processing order affects the number of processed samples that interleaves data dependant calculations. Thus, careful selection of processing order may alleviate the problem of data dependencies for pipelined hardware implementations. For the three main processing orders, the number of interleaving samples for $z-1$ and $t-1$ dependencies is summarized in Table 2.2, and may be calculated as follows:

1. BSQ iterates samples in the order (x, y, z). This means that $N_x \cdot N_y$ samples interleave $z-1$ dependencies as all samples of the first band are processed before moving to the next band. Spatial samples are processed sequentially, leaving a single sample between $t-1$ dependencies.
2. BIP iterates samples in the order (z, x, y), leaving a single sample between $z-1$ dependencies. All samples of the same pixel are processed before moving the next spatial index, giving N_z samples between $t-1$ dependencies.
3. BIL iterates samples in the order (x, z, y). Lines are processed sequentially, leaving N_x interleaved samples for $z-1$ dependencies and a single for $t-1$.

Table 2.2: Number of samples interleaving $z-1$ and $t-1$ dependencies for the three main processing orders.

	BSQ	**BIP**	**BIL**
$z-1$	$N_x \cdot N_y$	1	N_x
$t-1$	1	N_z	1

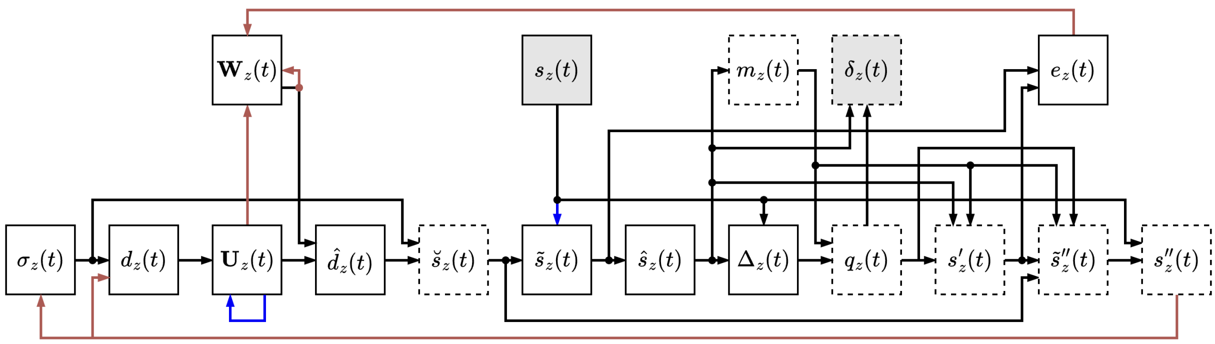


Figure 2.9: Data dependencies of the issue 2 predictor with $t-1$ and $z-1$ dependencies marked in red and blue respectively. Novel stages as compared to issue 1 are marked with stippled lines.

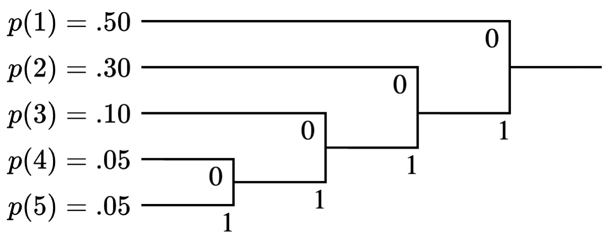
Figure 2.9 gives an overview of the data dependencies present in the issue 2 predictor, with $z-1$ and $t-1$ dependencies marked in blue and red respectively. Novel features compared to issue 1, mainly revolving around the quantization and sample representatives, are marked with stippled lines. In addition to the aforementioned dependency, weight updates form $t-1$ dependencies on the local difference vector $\mathbf{U}_z(t)$ and the double-resolution prediction error $e_z(t)$ (Equation 2.16). The latter depends on results from both the prediction calculation and sample representative calculation (Equation 2.17), presenting a particularly long path for the dependency to resolve compared to the shorter path of issue 1.

For the wide local sum configurations (Figure 2.7), calculation of local sums $\sigma_z(t)$ form a $t-1$ dependency on sample representatives $s''_z(t)$ (Equation 2.9). Furthermore, directional local differences have a $t-1$ dependence on sample representatives during full prediction mode. The local difference vector $\mathbf{U}_z(t)$ depends on local differences of the prediction bands, forming a $z-1$ dependency. Finally, the double-resolution predicted sample value has a $z-1$ dependency on the sample value from the previous band (Equation 2.21).

2.1.2 Entropy Coder

The mapped quantizer indices produced by the predictor are passed to an entropy coder where they are losslessly encoded and appended to the compressed bitstream. Entropy coders use sample statistics combined with predefined, variable-length codewords to encode samples with high probability with fewer bits than low probability samples. In this way, samples may be encoded using fewer bits than would be required if encoding the samples at face value.

A commonly used form of entropy encoding is *Huffman coding*. When the probabilities p of a finite set of messages are known, the method composes a set of codewords with a minimized average length [12]. A binary tree is constructed by recursively joining messages of lowest probability, as shown on the left side of Figure 2.10, producing the codewords of the accompanying table. As an example, the string of messages (denoted by index) $\{2, 3, 1, 1, 2, 5\}$ may be encoded using the string of codewords $\{10, 110, 0, 0, 10, 1111\}$, giving the bitstream 1011000101111. Codewords are prefix-free, meaning that no codeword is the prefix of another. This way, an encoded bitstream can be uniquely decoded by traversing from the start of the stream.



	Codeword
$p(1) = .50$	0
$p(2) = .30$	10
$p(3) = .10$	110
$p(4) = .05$	1110
$p(5) = .05$	1111

Figure 2.10: Huffman encoding tree and associated codeword table. Decoding of the prefix free codes may be performed by traversing from the top of the tree until a match is found at one of its branches. Figure recreated from [12].

Issue 1 supports two different entropy coders: the sample-adaptive and block-adaptive encoders. The former uses variable-length codewords, whereas the latter uses a different technique which will not be covered in this work. The minimum length of a variable-length codeword is one bit, meaning that encoding bitrates of less than one bit per sample are unachievable using the sample-adaptive encoder. Due to the low entropy of mapped quantizer indices when using the issue 2 near-lossless mode, a new entropy coder was introduced. This encoder, the *Hybrid encoder*, is able to achieve sub-one bitrates using a combination of the techniques of the sample-adaptive encoder and Variable-to-Variable (V2V) codes. The following overview of the hybrid encoder is based on the work presented by Blanes et al., which gives great insight into the development of and inner workings of the encoder [13].

Hybrid Encoder

Similarly to the sample-adaptive encoder, the hybrid encoder uses adaptive statistics to select between encoding techniques. By adding processed samples to an accumulator while keeping count of the number of processed samples, an estimate of the average sample value may be obtained by dividing the accumulator by the counter. At certain intervals the accumulator and counter are divided by two. This reduces the impact of previously processed samples while preserving the average estimate. Users may change the length of this interval by use of the rescaling counter size γ^* parameter.

The high-resolution accumulator $\tilde{\Sigma}_z(t)$ keeps a separate accumulator for each band, whereas the counter $\Gamma(t)$ is shared between bands. They are given by

$$\tilde{\Sigma}_z(t) = \begin{cases} \tilde{\Sigma}_z(t-1) + 4\delta_z(t), & \Gamma(t-1) < 2^{\gamma^*} - 1, \\ \left\lfloor \dfrac{\tilde{\Sigma}_z(t-1) + 4\delta_z(t) + 1}{2} \right\rfloor, & \Gamma(t-1) = 2^{\gamma^*} - 1, \end{cases} \tag{2.26}$$

and

$$\Gamma(t) = \begin{cases} \Gamma(t-1)+1, & \Gamma(t-1) < 2^{\gamma^*}-1, \\ \left\lfloor \dfrac{\Gamma(t-1)+1}{2} \right\rfloor, & \Gamma(t-1) = 2^{\gamma^*}-1. \end{cases} \tag{2.27}$$

Based on the average estimate $\tilde{\Sigma}_z(t)/\Gamma(t)$, the hybrid encoder chooses between two different types of encodings: high-entropy and low-entropy codes. For $t = 0$, the mapped quantizer index is encoded as is using an unsigned integer of D bits. For $t > 0$, if $\tilde{\Sigma}_z(t) \cdot 2^{14} \geq T_0 \cdot \Gamma(t)$, a high-entropy code is selected. Otherwise, a low-entropy code is used. Here, T_0 is the low-entropy threshold.

Low-entropy codes use a set of 16 V2V codes, indexed by the code index i. The work by Blanes et al. shows that the statistical distribution of low-entropy mapped quantizer indices produced during near-lossless mode deviate from the geometric distribution of high-entropy indices. To accommodate this, a set of V2V where developed, allowing efficient encoding of the low-entropy codes. Each of the 16 codes has a corresponding codeword table, similar to the Huffman code example of Figure 2.10.

By use of the average estimate, and a threshold T_i, the low-entropy code index i is selected. When multiple indices of the same index are observed, possibly interleaved by high-entropy codes, they are accumulated (active prefix) until a matching codeword is found and appended to the bitstream. This allows multiple mapped quantizer indices to be encoded by a single codeword. Low-entropy codewords corresponding to multiple samples may appear in the bitstream several samples after the first sample it encodes. For this reason, decoding can no longer proceed in the forward direction. To allow backwards decoding, the V2V codes are suffix-free, rather than prefix-free. This means that no codeword is the suffix of another.

High-entropy indices are encoded using a set of 29 Golomb power-of-two (GPO2) codes. First described by Golomb in [14], Golomb codes device a method for producing optimal prefix-free codewords for a set of messages following a geometric distribution. By using parameters which are powers of two, thereby Golomb *power-of-two*, operations required by hardware implementations are simplified at a slight loss in encoding performance. Additionally, the maximum codeword length is limited to $U_{max} + D$ bits, where U_{max} is the unary length limit, to further simplify hardware and cost of encoding outliers. To facilitate the backwards decoding required by low-entropy codes, the GPO2 are reversed to produce suffix-free codes.

In order to retrieve the mapped quantizer indices from the compressed bitstream, the decoder must be able to reproduce the average estimates of the encoder. For this reason, a compressed image tail encodes the final accumulator values and state of the V2V active prefixes. During encoding, the average estimate is updated before encoding the sample. This allows the decoder to use the current average estimate to select high- or low-entropy decoding before updating the estimate.

2.1.3 Header

2.2 Hardware Accelerators [1]

Embedded real-time systems often operate under strict deadlines. Missing these deadlines can be detrimental to system performance and correct functioning. At the same time, designers must balance budgets on power, area and cost. An effective approach to designing robust and responsive systems is to utilize multiprocessors [15]. Multiprocessor systems contain multiple Processing Elements (PE), that can be optimized for specific functions, performing tasks in parallel. It is well known that the use of simpler units running in parallel can provide the same performance at a lower power and cost [15, 16].

In addition to programmable PEs, e.g. Central Processing Units (CPU), which provide great flexibility, multiprocessor systems can achieve enormous performance improvements by the use of

[1]The section on hardware accelerators is taken as is from project thesis (TODO: make proper cite)

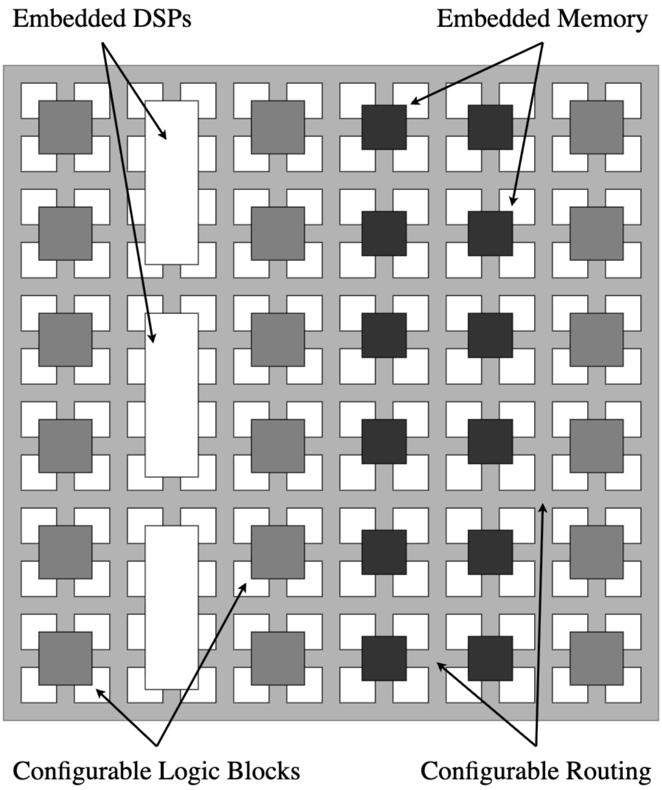


Figure 2.11: General layout of an FPGA. CLBs, DSPs, and memory blocks are placed in a configurable routing network.

specialized hardware accelerators [15]. One such example is the Graphical Processing Unit (GPU), which has become a staple of modern personal computers. FPGAs allow designers to design custom logic at low costs, as compared to the higher performance and costly Application Specific Integrated Circuits (ASIC). Most FPGAs also come at the added benefit of being reconfigurable, which provides flexibility for remotely deployed systems.

Figure 2.11 shows the layout of a typical FPGA. By use of Look-Up-Tables (LUT) and Flip-Flops (FF), Configurable Logic Blocks (CLB) can be configured to provide any logic function. Units are mounted in a large routing matrix, allowing custom routing between the CLBs. These two mechanisms provide the core functionality of the FPGA, and allow the implementation of any logic design. In order to increase efficiency in area, power and performance, it is common to add additional, specialized hardware units. Digital Signal Processors (DSP) provide common arithmetic functions, while embedded Block Random Access Memory (BRAM) provides efficient data storage. The hardware resource utilization of a design for FPGA is typically measured in terms of the amount of LUTs, FFs, DSPs, and BRAM used.

Another way of utilizing parallelization to increase performance is pipelining. By separating data independent stages of the calculation with registers, known as pipeline registers, the clock frequency can be increased while still issuing new data for processing every cycle. This approach is widely used in hardware designs, for example in CPUs. The clock frequency is limited by the longest path between any two pipeline registers. For implementations of the issue 2 algorithm, the aforementioned data dependencies create problems when trying to achieve high degrees of pipelining.

Compared to software design, hardware design comes at increased complexity in terms of design and development. A typical design starts at a microarchitectural level before the design is converted to the Register Transfer Level (RTL) using languages like VHDL, Verilog and Systemverilog. RTL code is then synthesized to a gatelevel netlist which, through a process called Place and Route (PnR), is mapped to a specific FPGA or for ASIC. To ensure correct functioning of the final result, extensive verification must be performed at all steps in the design process.

To alleviate the challenges in long design time of RTL code, a new design flow has emerged named High-Level Synthesis (HLS). The advantage of using the HLS approach is reduced development time, as the algorithm can be described in a high-level language like C or C++, which is then converted to RTL [17]. However, the tools for such a conversion create suboptimal results when compared to writing the implementation directly in RTL using languages like VHDL or Systemverilog, giving the designer fine-grained control over optimizations.

The satellites of the HYPSO project use Multiprocessor System-on-Chips (MPSoC) from AMD (formerly Xilinx), namely the Zynq chips [3]. These COTS embed two ARM processing units, a simple GPU, and several useful peripherals including Analog-to-Digital Converters (ADC), timers and support for various communication protocols. The system runs the Linux operating system, providing tools for management of complex tasks and threads. Additionally, the chips include embedded Programmable Logic (PL), equivalent to an embedded FPGA. Embedding the PL directly in the MPSoC allows custom logic to directly access the main data buss and other systems on the chip, including Direct Memory Access (DMA) for fast and CPU independent memory accesses. These features give great benefits in terms of efficiency in speed and power as compared to separate FPGAs.

TODO: paragraph about verification. DUT, constrained random, golden model etc

2.3 Coding Performance and Decompressed Image Quality

$$\mathrm{CR} = \frac{N_x \cdot N_y \cdot N_z \cdot D}{\text{compressed data size (bits)}}. \tag{2.28}$$

$$\mathrm{PSNR}_z = 10 \cdot \log_{10}\left(\frac{\mathrm{MAX}_I^2}{\mathrm{MSE}_\mathrm{z}}\right), \tag{2.29}$$

$$\mathrm{MSE}_\mathrm{z} = \frac{1}{N_x N_y} \sum_{i=0}^{N_x-1} \sum_{j=0}^{N_y-1} \left[I_z(i,j) - K_z(i,j) \right]^2. \tag{2.30}$$

$$\mathrm{PSNR}_{HSI} = \frac{1}{N_z} \sum_{z=0}^{N_z-1} \mathrm{PSNR}_z. \tag{2.31}$$

2.4 Software Optimization

Compiled vs interpreted languages. Amdals law

Chapter 3

State-of-the-Art

Table 3.1: The number of processed samples interleaving $z-1$ and $t-1$ dependencies, including the novel FID ordering in addition to the standard orderings.

	BSQ	BIP	BIL	FID
$z-1$	$N_x \cdot N_y$	1	N_x	$N_z - 1$
$t-1$	1	N_z	1	N_z

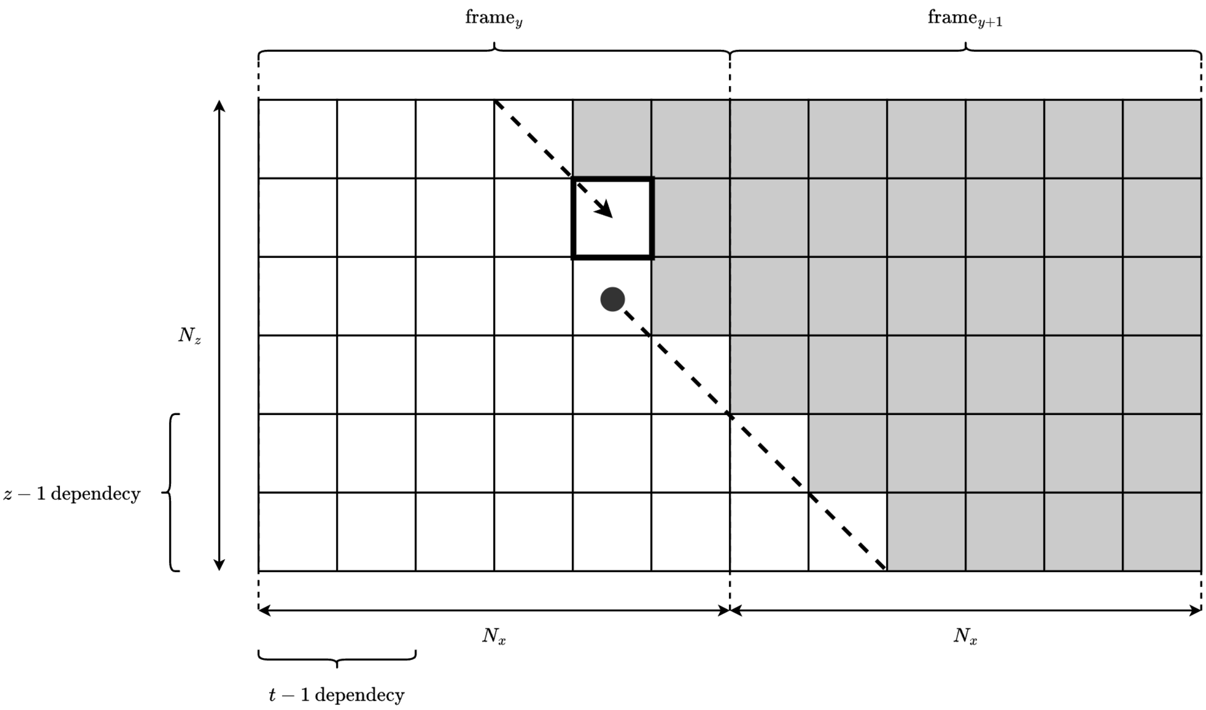


Figure 3.1

$$\mathrm{sgn}^+\left[e_z(t)\right] = \begin{cases} -1, & \text{if } |s_z(t) - \hat{s}_z(t)| \leq m_z(t) \text{ and } \tilde{s}_z(t) \text{ is odd,} \\ 1, & \text{if } |s_z(t) - \hat{s}_z(t)| \leq m_z(t) \text{ and } \tilde{s}_z(t) \text{ is even,} \\ \mathrm{sgn}^+\left[s_z(t) - \hat{s}_z(t)\right], & \text{otherwise.} \end{cases} \tag{3.1}$$

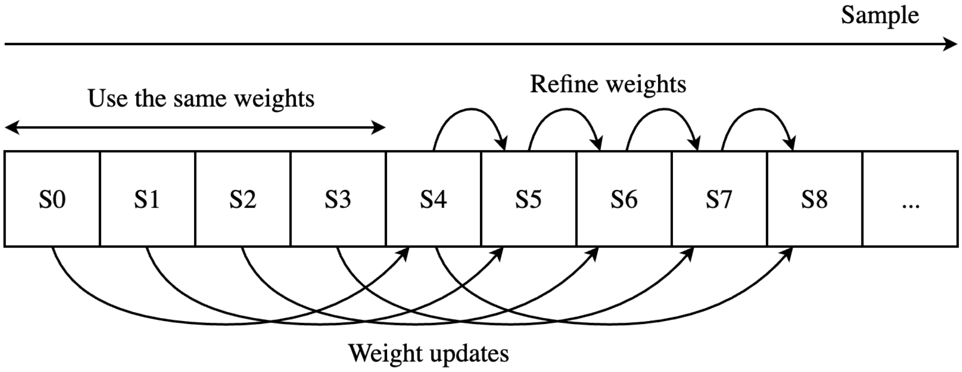


Figure 3.2: The proposed weight update scheme by Jia et al. Weights are kept unchanged for the first samples of every line, after which the sample from four samples earlier is used in the weight update calculation.

$$\omega^{(i)}_{z,\text{unclipped}}(t+1) = \begin{cases} \omega^{(i)}_z(t), & x \leq 2, \\ \omega^{(i)}_z(t-3) + \omega^{(i)}_{z,\text{offset}}(t-3), & x = 3, \\ \left\lfloor \frac{1}{2}\left(\omega^{(i)}_z(t-3) + \omega^{(i)}_{z,\text{offset}}(t-3) + \omega^{(i)}_z(t)\right) \right\rfloor, & \text{otherwise.} \end{cases} \tag{3.2}$$

Chapter 4

Previous Work

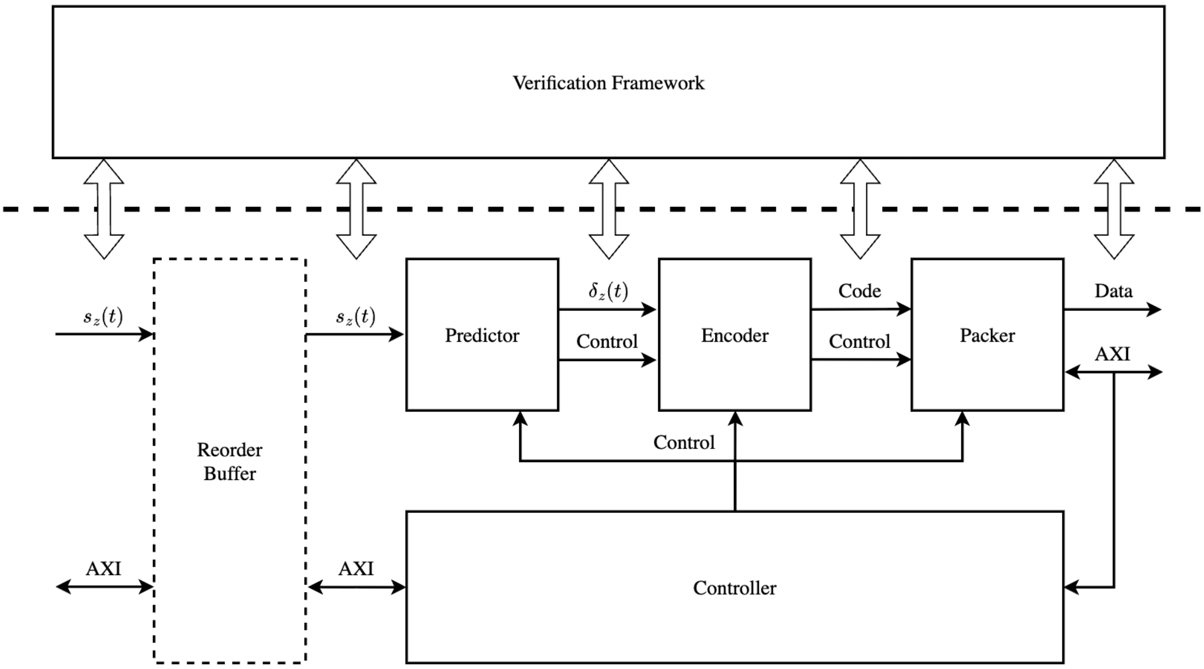


Figure 4.1: Block diagram of Vorhaug. For easy integration into HYPSO, a reorder buffer reorders samples from BIP to BIL.

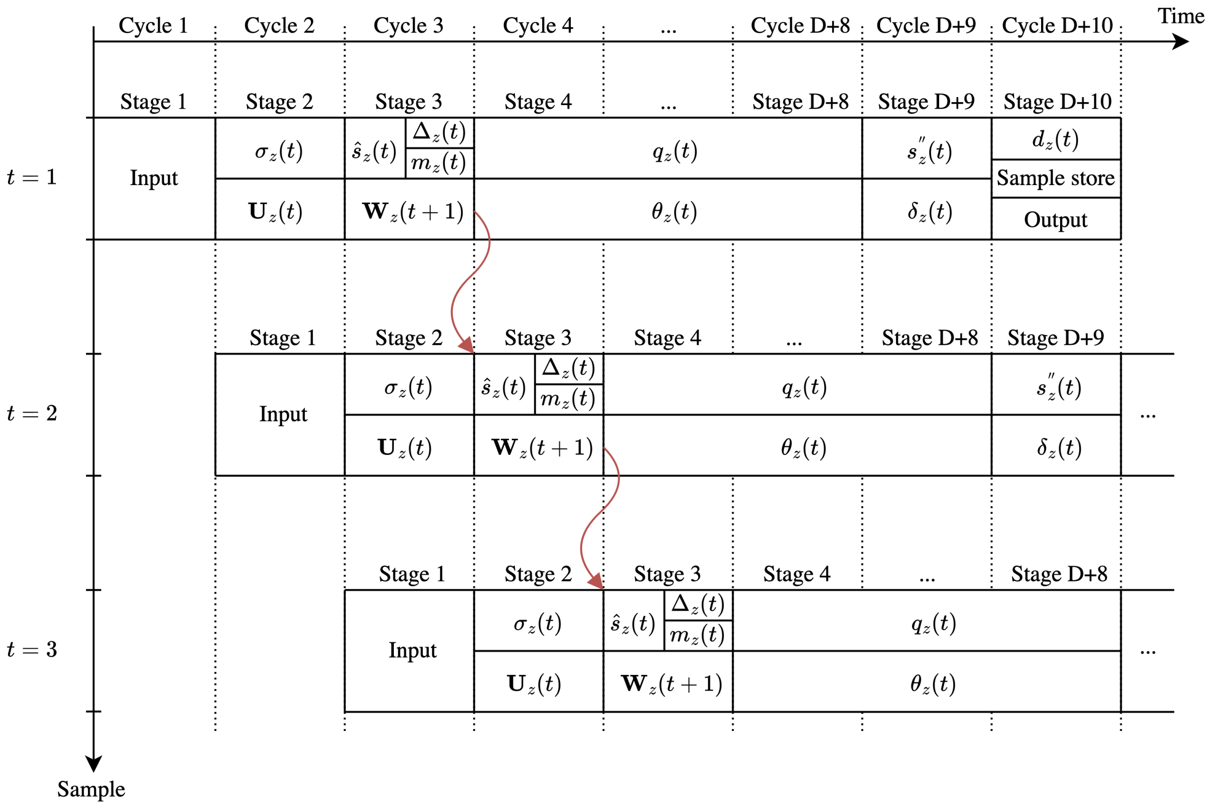


Figure 4.2: Full pipeline of Vorhaug. The indivisible weight update calculation in stage 3 restricts the throughput.

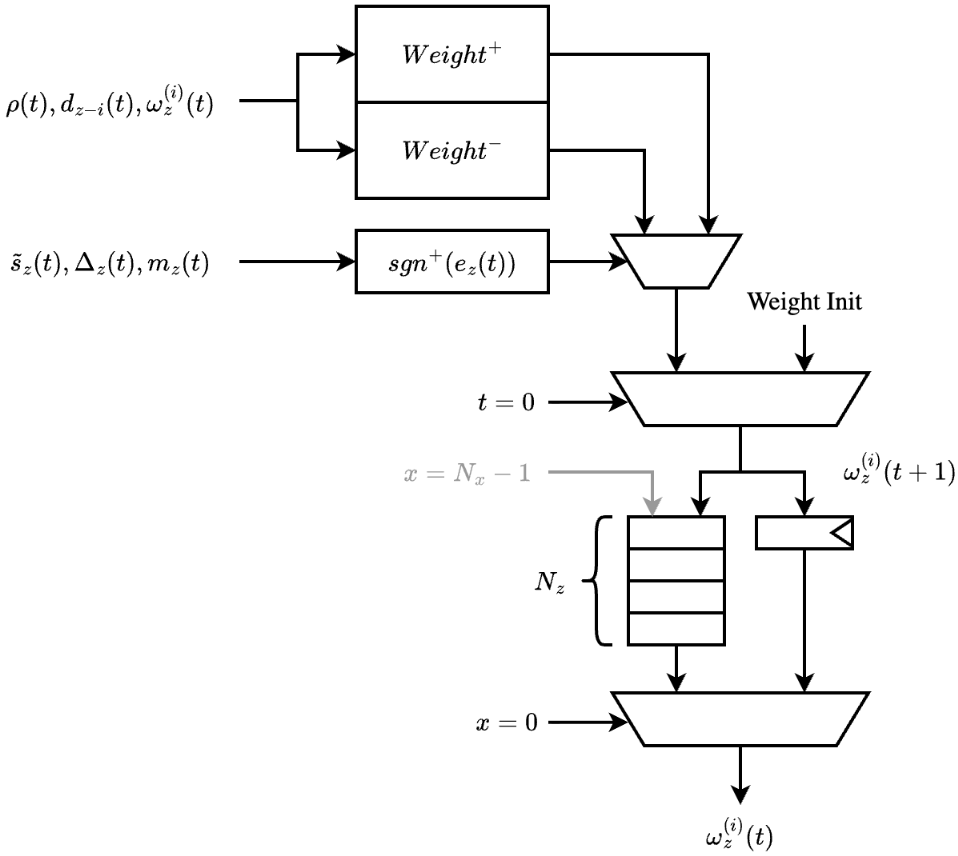


Figure 4.3: Weight update stage of Vorhaug following the mathematical optimization approach proposed by Sánchez et al. Weight updates are calculated speculatively while the sign of the prediction error resolves.

Chapter 5

Implementation and Verification

5.1 Verification Model

Predictor with compression and decompression implementation in C++.

Reconstruction of mapped quantizer indices from compressed bitstream, hybrid encoder.

5.2 Hardware Modifications

5.3 Hardware Verification

5.4 Hardware Testing

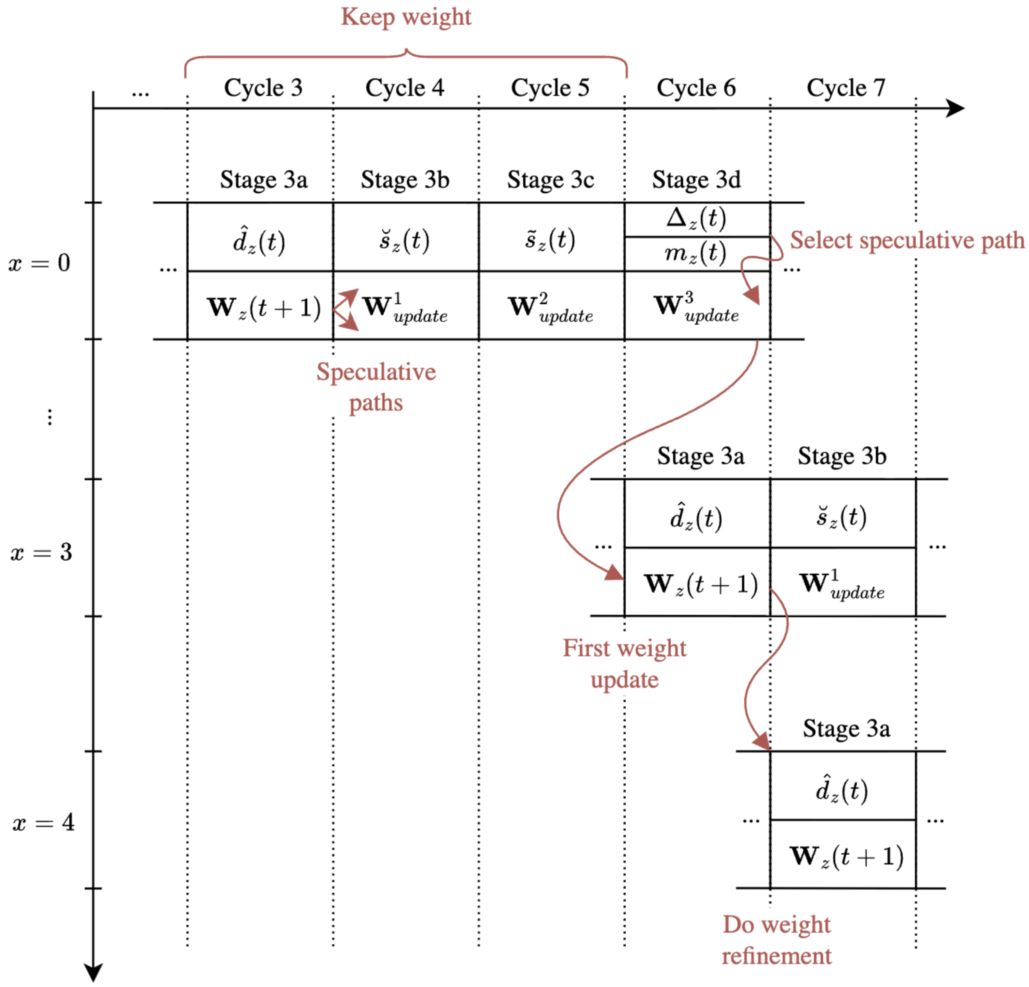


Figure 5.1: Proposed pipeline for delayed weight updates.

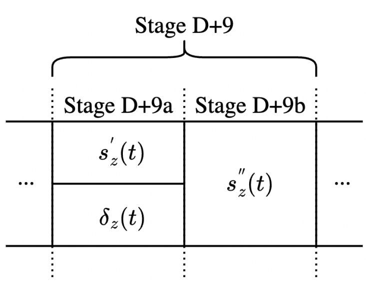


Figure 5.2: Sample representative stage split into two stages $D+9a$ and $D+9b$, reducing the new critical path after splitting the weight update stage.

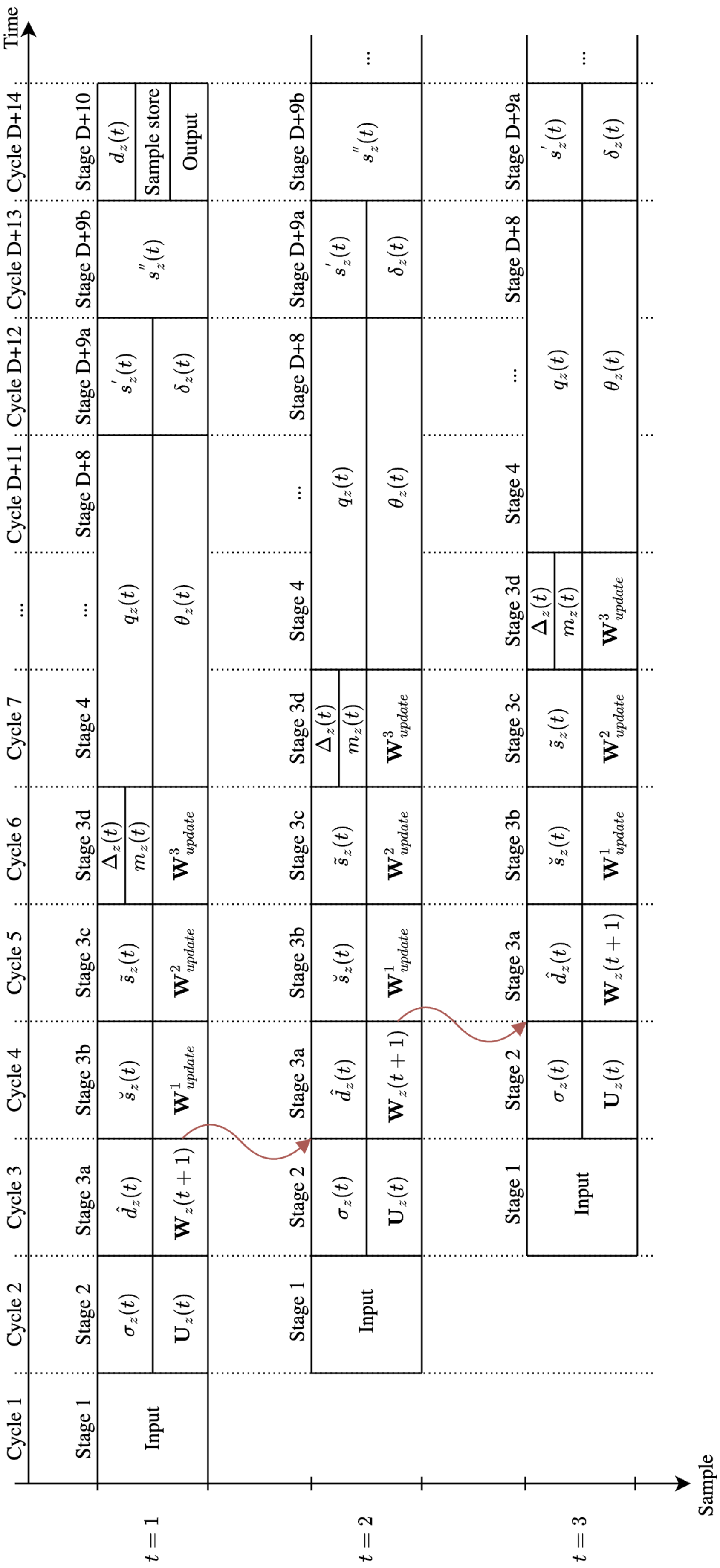


Figure 5.3: Proposed pipeline with delayed weight updates allowing splitting stage 3 into four separate stages, 3a to 3d. Additionally, the sample representative calculation in stage $D+9$ is split into two stages $D+9a$ and $D+9b$.

Table 5.1: Selected predictor and hybrid encoder parameters. Values are selected based on the work by Blanes et al., and are the same as those used by Vorhaug [5, 19]. Table recreated from [5].

Name	**Symbol**	**Configuration**
Sample type		Unsigned
Dynamic range	D	16
Sample encoding order		BIL
Entropy coder type		Hybrid coder
Quantizer method		Absolute
Number of prediction bands	P	3
Register size	R	64
Local sum type		Narrow column-oriented
Prediction mode		Reduced
Weight component resolution	Ω	19
Weight initialization method		Default
Weight update initial parameter	ν_{min}	-1
Weight update final parameter	ν_{max}	4
Weight update change interval	t_{inc}	2^6
Weight update exponent offsets	$\zeta_z^{(i)}, \zeta_z^*$	0
Periodic error limit update		Disabled
Band-varying absolute limits		Disabled
Absolute error limit	a_z	*Varied*
Band-varying relative limits		Disabled
Relative error limit	r_z	0
Sample representative resolution	Θ	3
Sample representative offset	ψ_z	7
Band-varying offset		Disabled
Sample representative damping	θ_z	3
Band-varying damping		Disabled
Unary length limit	U_{max}	18
Rescaling counter size	γ^*	6
Initial count exponent	γ_0	1
Accumulator initialization value	$\tilde{\Sigma}_z(0)$	$4 \cdot 2^{\gamma_0}$

Chapter 6

Results

6.1 Verification Model

Table 6.1: Compression performance, execution time of predictor in parenthesis (TODO: find better way of displaying than in parenthesis)

Image	Predictor	Samples	Time (s)		Throughput (MS/s)	Memory (MB)
Hypso 2	C++	78361920	821	(293)	0.095	32415
Aviris	C++	77643776	1166	(290)	0.066	32091
Landsat	C++	6291456	103	(22)	0.061	2669
Hypso 2	Python	78361920	2818	(2169)	0.027	38996
Landsat	Python	6291456	263	(161)	0.024	3204

Table 6.2: Decompression

Image	Samples	Time (s)	Throughput (MS/s)	Memory (MB)
Hypso 2	78361920	1093	0.072	32414
Aviris	77643776	1834	0.042	32091
Landsat	6291456	149	0.042	2667

6.2 Compression Performance

Table 6.3: Delayed weight updates, relative 16 and absolute 4

Image	Delayed	Ordinary	Decrease
Hypso 2	2.47	2.49	−0.80%
Aviris	4.31	4.41	−2.23%
Landsat	4.82	5.21	−7.49%

Table 6.4: Reductions in compression performance for coastline image for this work and the work by Jia et al.

PAE	Coastline, Jia et al.	Coastline, this work
0	−1.53%	−0.50%
1	−2.88%	−0.78%
3	−3.86%	−0.63%
7	−4.30%	−0.98%
15	−1.23%	−1.10%

Table 6.5: PSNR of decompressed images with delayed weight updates. Difference compared to without delayed updates are degligable, exact same when rounded to two decimals.

PAE	CR	PSNR (dB)
0	2.02	N/A
1	2.53	225.86
3	3.15	207.94
7	4.03	192.54
15	5.39	178.10

6.3 RTL Results

Table 6.6

FPGA	LUT	FF	DSP	BRAM	Power	Frequency
Zynq-7030	5958	3425	4	83.5	152 mW	90.9 MHz
Zynq ZU7EV	6331	3442	4	79.5	273 mW	200.0 MHz
Zynq-7030 (predictor)	2841	2212	4	69.5	263 mW	125.0 MHz
Zynq ZU7EV (predictor)	2969	2185	4	65.5	467 mW	312.5 MHz

6.4 Hardware Testing

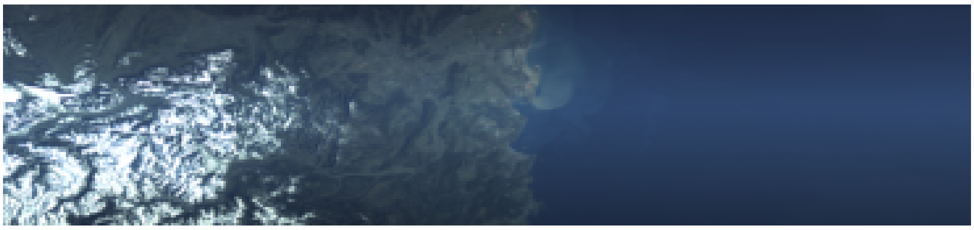


Figure 6.1: Decoded Hypso 2 image

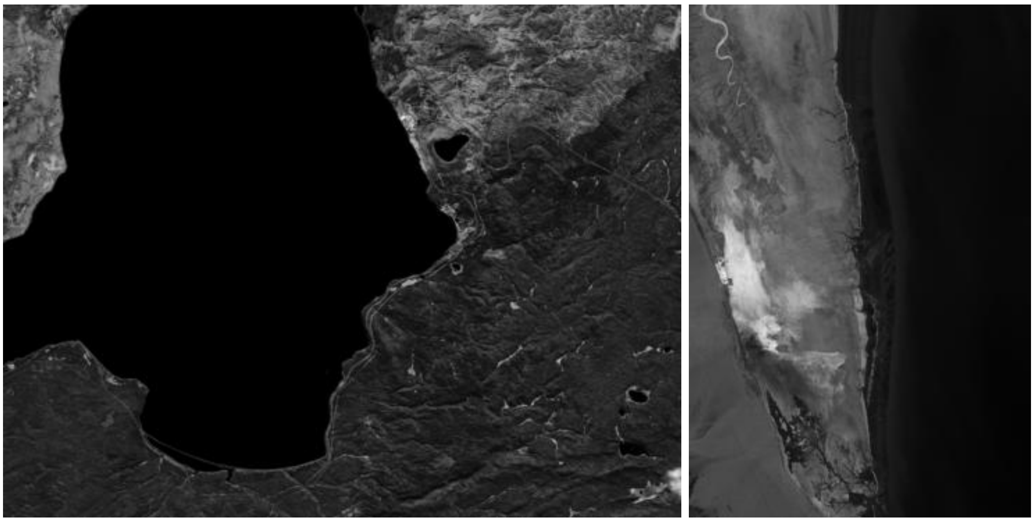


Figure 6.2: Decoded Aviris and Landsat images.

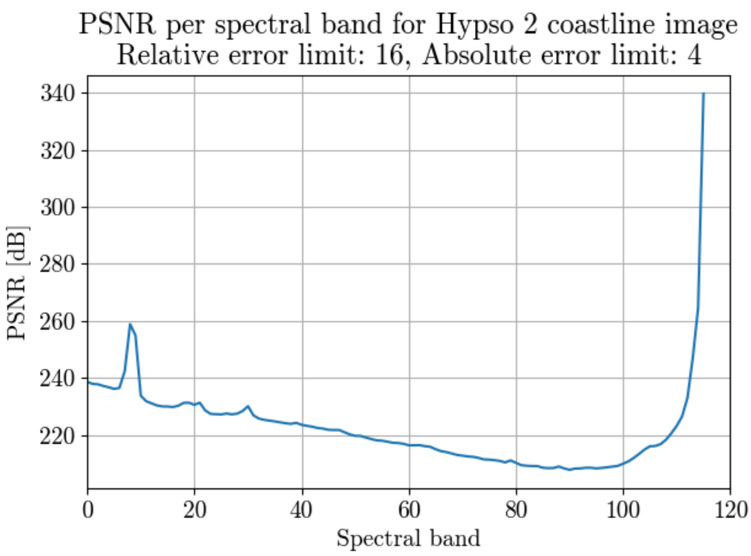


Figure 6.3

Chapter 7

Discussion and Further Work

Work should be done to properly integrate the issue 2 implementation into the HYPSO-2 on-board-processing (OPU) system. This includes writing software for header, and selection of parameters. As of now, HYPSO-2 still uses the outdated issue 1 implementation. Although fast, it cannot achieve compression rates close to those of issue 2. Issue 2 for HYPSO-3!!

Chapter 8

Conclusions

Bibliography

[1] S. Bakken, M. B. Henriksen, R. Birkeland, D. D. Langer, A. E. Oudijk, S. Berg, Y. Pursley, J. L. Garrett, F. Gran-Jansen, E. Honoré-Livermore, M. E. Grøtte, B. A. Kristiansen, M. Orlandic, P. Gader, A. J. Sørensen, F. Sigernes, G. Johnsen and T. A. Johansen, 'HYPSO-1 CubeSat: First Images and In-Orbit Characterization,' *Remote Sensing*, vol. 15, no. 3, Jan. 2023, ISSN: 2072-4292. DOI: 10.3390/rs15030755.

[2] E. J. Ientilucci and S. Adler-Golden, 'Atmospheric Compensation of Hyperspectral Data: An Overview and Review of In-Scene and Physics-Based Approaches,' *IEEE Geoscience and Remote Sensing Magazine*, vol. 7, no. 2, Jun. 2019, ISSN: 2168-6831. DOI: 10.1109/MGRS.2019.2904706.

[3] M. E. Grøtte, R. Birkeland, E. Honoré-Livermore, S. Bakken, J. L. Garrett, E. F. Prentice, F. Sigernes, M. Orlandić, J. T. Gravdahl and T. A. Johansen, 'Ocean Color Hyperspectral Remote Sensing With High Resolution and Low Latency—The HYPSO-1 CubeSat Mission,' *IEEE Transactions on Geoscience and Remote Sensing*, vol. 60, 2022, ISSN: 1558-0644. DOI: 10.1109/TGRS.2021.3080175.

[4] L. Jia, Q. Wang, L. Zhang, C. Song and P. Zhang, 'Removal of the Feedback Loop in CCSDS 123.0-B-2 during Hardware Implementation,' *IEEE Geoscience and Remote Sensing Letters*, 2025, ISSN: 1545-598X, 1558-0571. DOI: 10.1109/LGRS.2025.3601194.

[5] D. Vorhaug, 'Hyperspectral Image Compression Accelerator On FPGA Using CCSDS 123.0-B-2,' M.S. thesis, NTNU, 2024.

[6] J. Fjeldtvedt, M. Orlandić and T. A. Johansen, 'An Efficient Real-Time FPGA Implementation of the CCSDS-123 Compression Standard for Hyperspectral Images,' *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, vol. 11, no. 10, Oct. 2018, ISSN: 2151-1535. DOI: 10.1109/JSTARS.2018.2869697.

[7] The Consultative Committee for Space Data Systems, 'Low-Complexity Lossless and Near-Lossless Multispectral and Hyperspectral Image Compression,' CCSDS, Recommended Standard CCSDS 123.0-B-2, 2019.

[8] M. Klimesh, 'Low-complexity lossless compression of hyperspectral imagery via adaptive filtering,' *Interplanetary Network Progress Report*, Jan. 2005.

[9] The Consultative Committee for Space Data Systems, 'Lossless Multispectral and Hyperspectral Image Compression,' CCSDS, Green Book Report CCSDS 120.2-G-2, 2022.

[10] D. Keymeulen, S. Shin, J. Riddley, M. Klimesh, A. Kiely, E. Liggett, P. Sullivan, M. Bernas, H. Ghossemi, G. Flesch, M. Cheng, S. Dolinar, D. Dolman, K. Roth, C. Holyoake, K. Crocker and A. Smith, 'High performance space computing with system-on-chip instrument avionics for space-based next generation imaging spectrometers (ngis),' Aug. 2018, pp. 33–36. DOI: 10.1109/AHS.2018.8541473.

[11] A. Gersho, 'Adaptive filtering with binary reinforcement,' *IEEE Transactions on Information Theory*, vol. 30, no. 2, pp. 191–199, Mar. 1984, ISSN: 1557-9654. DOI: `10.1109/TIT.1984.1056890`. Accessed: 5th Dec. 2025. [Online]. Available: `https://ieeexplore.ieee.org/abstract/document/1056890`.

[12] J. W. Woods, 'Digital Image Compression,' in *Multidimensional Signal, Image, and Video Processing and Coding*, Elsevier, 2012, pp. 329–392, ISBN: 978-0-12-381420-3. DOI: `10.1016/B978-0-12-381420-3.00009-6`. Accessed: 3rd Mar. 2026. [Online]. Available: `https://linkinghub.elsevier.com/retrieve/pii/B9780123814203000096`.

[13] I. Blanes, A. Kiely, L. Santos, M. Hernández-Cabronero and J. Serra-Sagristà, 'THE HYBRID ENTROPY ENCODER OF CCSDS 123.0-B-2: INSIGHTS AND DECODING PROCESS,'

[14] S. Golomb, 'Run-length encodings (Corresp.),' *IEEE Transactions on Information Theory*, vol. 12, no. 3, pp. 399–401, Jul. 1966, ISSN: 1557-9654. DOI: `10.1109/TIT.1966.1053907`. Accessed: 8th May 2026. [Online]. Available: `https://ieeexplore.ieee.org/document/1053907/`.

[15] M. Wolf, 'Chapter 10 - embedded multiprocessors,' in *Computers as Components (Fifth Edition)*, ser. The Morgan Kaufmann Series in Computer Architecture and Design, M. Wolf, Ed., Fifth Edition, Morgan Kaufmann, 2023, pp. 453–484. DOI: `10.1016/B978-0-323-85128-2.00010-4`.

[16] R. Sarpeshkar, 'Universal Principles for Ultra Low Power and Energy Efficient Design,' *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 59, no. 4, Apr. 2012, ISSN: 1558-3791. DOI: `10.1109/TCSII.2012.2188451`.

[17] Y. Barrios, A. Sánchez, R. Guerra and R. Sarmiento, 'Hardware Implementation of the CCSDS 123.0-B-2 Near-Lossless Compression Standard Following an HLS Design Methodology,' *Remote Sensing*, vol. 13, no. 21, p. 4388, Jan. 2021, ISSN: 2072-4292. DOI: `10.3390/rs13214388`.

[18] A. Sánchez, I. Blanes, Y. Barrios, M. Hernández-Cabronero, J. Bartrina-Rapesta, J. Serra-Sagristà and R. Sarmiento, 'Reducing Data Dependencies in the Feedback Loop of the CCSDS 123.0-B-2 Predictor,' *IEEE Geoscience and Remote Sensing Letters*, vol. 19, 2022, ISSN: 1558-0571. DOI: `10.1109/LGRS.2022.3213975`.

[19] I. Blanes, A. Kiely, M. Hernández-Cabronero and J. Serra-Sagristà, 'Performance Impact of Parameter Tuning on the CCSDS-123.0-B-2 Low-Complexity Lossless and Near-Lossless Multispectral and Hyperspectral Image Compression Standard,' *Remote Sensing*, vol. 11, no. 11, Jan. 2019, ISSN: 2072-4292. DOI: `10.3390/rs11111390`.

Appendix A

Block Design

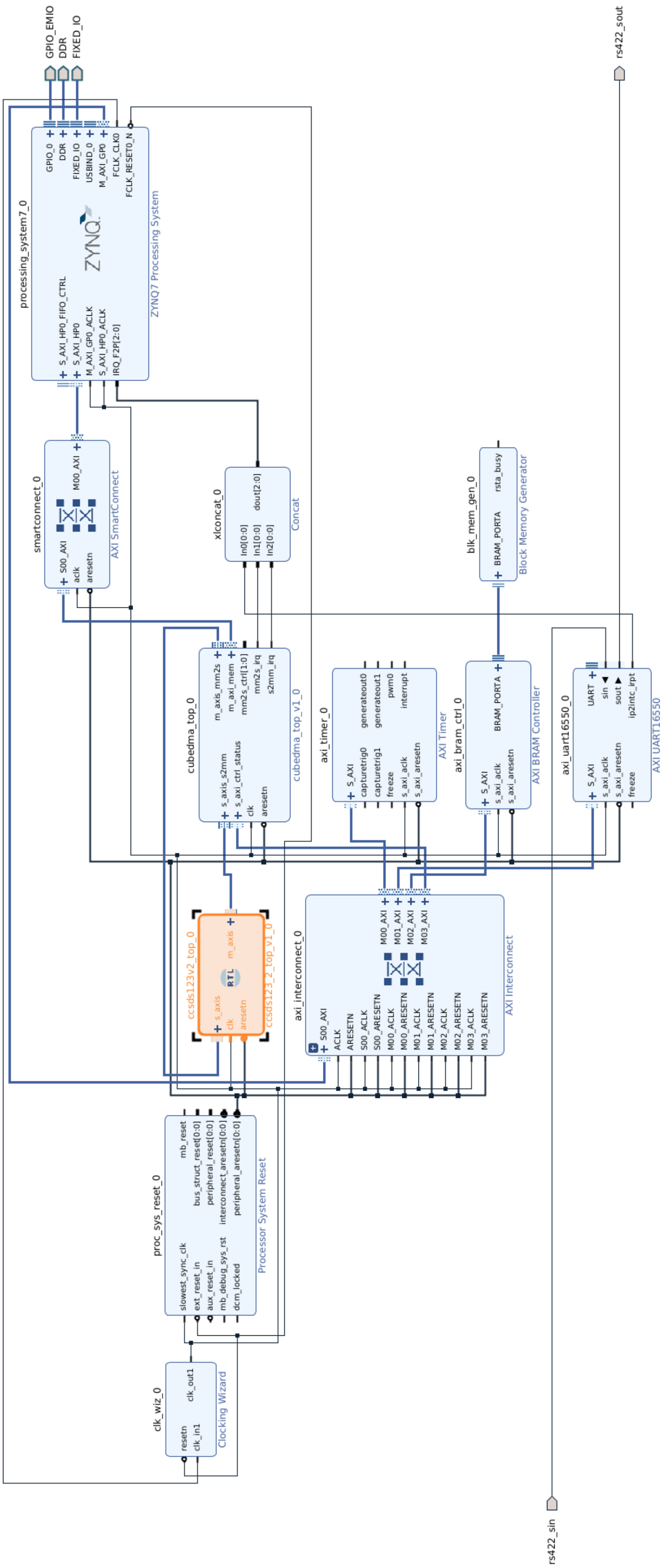


Figure A.1